

EMUQU — Enforced Scientific Honesty: Build-Time Governance of a Consumer Physiology Score

Constants-hash version locks, a machine-checked science register, gates that must prove they fail, and one-way debt ratchets

Technical White Paper • v1.2

v1.2 (2026-10-08): restructured method-first; adds design rationale; validation detail moved to the separate validation report.

October 2026

Chris Sharp

Describes Emuqu at source revision e028039 (2026-10-07); scoring version v3.1.oct2026.

Executive Summary

Emuqu is an iOS app that turns overnight beat-to-beat (RR) intervals from a chest strap, with sleep and vitals data, into a daily recovery score. Like every consumer composite score, its weights and bands are hand-calibrated: some inputs rest on published methods, others are product rules whose status the app records openly. The app is not a medical device. Keeping such a score honest over time means preventing four kinds of quiet drift:

1. A scoring constant changes, and stored history becomes a mixture of two algorithms under one name.
2. A heuristic's evidence status is forgotten, and user-facing copy starts to imply more than the evidence supports.
3. A quality gate reports "clean" while measuring nothing.
4. A debt ceiling is raised to make the build pass instead of paying the debt.

Emuqu answers each of these with build-time machinery, all of it plain bash and Python run in continuous integration (CI):

- **A constants-hash version lock.** The numeric constant declarations in two designated scoring files (155 at this revision) are reduced to a sorted `file:symbol=value` list, hashed with SHA-256 and truncated to 16 hex characters. The build fails if that digest moves while the scoring-version string still equals the recorded version. Every stored score breakdown carries its version, and records from before versioning decode to an explicit `unversioned` sentinel.
- **A machine-checked science register.** A JSON inventory classifies 19 heuristics into a closed status vocabulary (3 `validated`, 4 `supported-transfer`, 12 `awaiting-validation`, 0 `scheduled-for-`

removal). Each entry states the wording user-facing copy may and may not use. A gate checks fields, cited test paths, version agreement with the code and retracted phrasings, and requires every score-adjusting function found by a name scan to be claimed or waived.

- **Gates that test the gates.** Each cheap gate must go red on a planted violation (64 plants), fail closed on temp-file errors, and be wired into CI. A separate harness applies 125 targeted production mutations that named test suites must catch, and a static backstop detects any mutation left in the tree.
- **One-way ratchets.** Each CI budget file may move only in its tightening direction, read from the filename. A loosening needs a `budget-raise-ok`: commit trailer, and docs may not restate a ratcheting value.

The parts form one chain. A change to a scoring constant forces a version change; the version change forces an edit of the evidence register, which states each heuristic's permitted wording; and that wording is policed for prohibited vocabulary by the copy perimeter of White Paper 3. Every gate in the chain is itself proven to go red, to fail closed and to be reached by CI.

The work builds on published tools for checking checks [19][20][21], requirement-status inventories [22][23][24] and coverage ratchets [25][26]. Section 4 states precisely what it adds, and Section 8 gives the dated record for each statement.

1. The Problem

1.1 Composite scores are calibrated, not validated

Consumer wearables publish composite "readiness" or "recovery" numbers built from heart-rate variability (HRV), sleep and other signals. The component measurements have a literature: the Task Force standards for time-domain HRV [1], and the smallest-worthwhile-change method on a rolling $\ln(\text{RMSSD})$ baseline [2]. The way they are combined usually does not. Reviews of consumer wearables in athletic populations note that the proprietary algorithms are rarely validated independently [3]. Some popular sub-heuristics have been challenged directly. The LF/HF ratio as an index of "sympathovagal balance" is one [4]. The acute:chronic workload ratio as an injury-risk signal is another [5]. Emuqu's own register records that its composite weights are calibrated, not outcome-validated.

A product in this position can be honest in two ways. It can remove every unvalidated rule, which in practice removes the product. Or it can keep bounded heuristics and make their status impossible to forget. Emuqu takes the second route. The second route works only if forgetting is mechanically prevented, and that is what this paper describes.

1.2 Sources of drift

Drift	How it happens	Why review alone misses it
Algorithm mixing	A threshold or weight is retuned; stored scores computed before and after look identical	The diff touches one number, and nothing marks history
Claim creep	Copy is reworded ("strong predictor" becomes "strongly suggests")	A lexical banned-word list matches phrasings, not claims

Drift	How it happens	Why review alone misses it
Dead evidence	A test cited as backing for a rule is deleted or renamed	Nobody re-reads the evidence inventory
Resurrected errors	A figure corrected in one file survives in another	Corrections are local; copies are not
Fail-open gates	A temp file cannot be created; the gate greps an empty file and prints "clean"	A green tick looks the same whether or not anything was measured
Unwired gates	A gate exists in the scripts directory but no CI step runs it	The directory listing looks complete
Goalpost moves	A budget file is edited upward so CI passes	The edit is one character

Each row is a failure a code reviewer reads straight past, because the change that causes it is small and locally reasonable. The common answer is to make each one a build failure.

1.3 What existing approaches provide

Emuqu builds on several established families of tools, each of which solves part of the problem. Lint baselines (SwiftLint's `--baseline`, betterer [6]) freeze existing violations so that new ones fail, and coverage ratchets (jest-ratchet [26], CoverageRatchet [25]) let a coverage threshold only rise. Model cards and model registries [7][8] attach documentation and version identifiers to machine-learning models. Mutation testing frameworks (PIT [9], Stryker [10]) mutate code broadly and report a mutation score. Requirements-traceability tools (sphinx-needs [22], Doorstop [24], reqcov [23]) keep requirements in the repository, restrict statuses to a configured list and link items to tests and code. Checks on checks (canfail [19], didrun [20] and restore-verified [21], all published on PyPI on 2026-08-31) require a check to go red on a deliberate break, to show evidence that it ran, and to restore mutated files verifiably. Medical-device quality systems (IEC 62304 [11], ISO 13485 [12]) require traceability and controlled change through audited process; Emuqu is not developed under them.

Emuqu adds the links between these: a numeric limit whose direction is declared per file, a build failure when a hand-set constant changes without a version change, a claim inventory tied to that version and to a per-heuristic wording boundary, mutations that each name the behaviour and test suite at stake, and completeness, so that every gate in the repository has a plant and is reached by CI. Section 4 states each addition against its closest prior art.

1.4 The approach

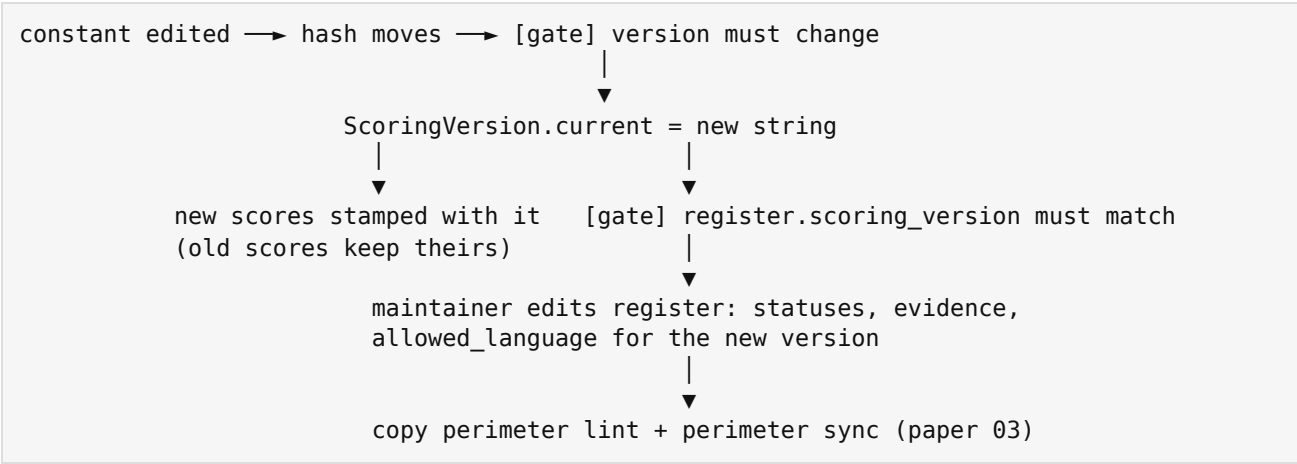
Emuqu binds three things together with executable checks: the numbers in the scoring code, the version string on stored scores, and the evidence status and permitted wording of each heuristic. Each binding is a gate, each gate is checked for the three properties that make a green result meaningful (it goes red, it fails closed, it is wired), and the numeric limits around the gates can only tighten. This build-time governance is the implementation-phase counterpart of the design method described in White Paper 5, blind-audit convergence with DO NOT FLAG lists: White Paper 5 governs how the design and its claims are reached, and this paper governs how they are kept true once the code exists.

2. Architecture Overview

The governance layer is a set of scripts in the repository, run in two CI workflows and by a local `make ci` target. None of it runs on the user's device. The on-device parts are the version stamp on stored scores and the text that shows it.

Phase	Operation	Purpose
Author time	Change a scoring constant	—
Gate: scoring governance	Hash numeric constants; compare with recorded <code>version</code> <code>hash</code> line	Force a version bump when the numbers move
Gate: scoring governance	Scan app sources for the ranking-weight identifier outside one module	Keep a hand-set ordering key from being displayed as a confidence
Runtime	Stamp every new score breakdown with the current version; decode unstamped history as <code>unversioned</code>	Keep each stored score attributable to its algorithm
Gate: science register	Register version must equal code version	Force an edit of the register on every version bump
Gate: science register	Status vocabulary, required fields, test paths, retracted claims, discovery scan, entry floor	Keep the claim inventory complete in a checkable sense and keep it current
Gate: copy perimeter (White Paper 3)	Prohibited-term lint on Swift string literals and string catalogues; build-time list reconciled with the runtime assistant guards	Keep medical-claim vocabulary out of user-facing text
Meta-gates	Planted violation per gate; fail-closed temp files; wiring check	Prove each gate goes red, fails closed and is reached
Test verification	Targeted mutation per behaviour; leak backstop; tree lock	Prove the test suites assert the behaviours that matter
Ratchets	Direction-aware budget comparison against a baseline commit	Debt only falls, floors only rise
Reporting	Evidence report prints live counts	Give reviewers numbers that cannot be stale

The chain that matters most for scientific honesty, shown as a sequence:



3. Methods in Detail

3.1 Scoring-constants hash and version lock

Inputs.

- Two designated scoring-constant source files: the recovery-score constants file (91 numeric declarations) and the HRV thresholds file (64 numeric declarations). That makes 155 at this revision.
- The scoring-version source file, which declares `current` and `unversioned`.
- A one-line baseline file in the CI directory containing `<recorded_version> <recorded_hash>`. At this revision it reads `v3.1.oct2026 541b56e600080795`.

Steps.

1. Visit the two constant files in a fixed order: recovery-score constants first, HRV thresholds second. If either file is missing, exit 2 (configuration error).
2. Read each file and delete every `//` line comment. The regex is `//[^\n]*`, replaced with nothing. Rewording a rationale comment therefore never changes the hash. Block comments are not stripped; neither file contains one.
3. Extract all matches of `static let\s+(\w+)\s*:\s*?\s*\s*[\w<>\\[\], .]*=\s*([-\.d.]+)` giving `(symbol, value)` pairs. `value` is the literal text of the number as written, so `60` and `60.0` are different values. A declaration whose right-hand side is not a bare numeric literal is not matched.
4. Sort the pairs of each file lexicographically by `(symbol, value)`. Moving a declaration within a file, or renaming an enclosing namespace, does not change the hash. Renaming a symbol or changing its value does. Symbols repeated under different nested namespaces are all included.
5. For each pair, in file order and then sorted order, feed the UTF-8 bytes of `"<file path>:<symbol>=<value>\n"` into one running SHA-256 digest.
6. Take the first 16 hexadecimal characters of the digest (64 bits) as `current_hash`.
7. Read `current` from the version file with `static let current\s*=\s*"([^\"]+)"`. If it cannot be read, exit 2.
8. If the baseline file is missing, exit 2. Otherwise split it on whitespace into `recorded_version` and `recorded_hash`.
9. **Violation rule:** if `current_hash != recorded_hash` **and** `version == recorded_version`, report "scoring constants changed while the version stayed the same" and exit 1.
10. Print the number of constants hashed, the hash and the version, whatever the result.

Why it works. A score's meaning is the set of numbers that produced it, so the version should change exactly when those numbers change. The hash captures the numbers and nothing else. Stripping comments means the rationale written beside a constant can be improved freely without a version bump. Sorting per file means reorganising code is free too, while a renamed or retuned constant is not. Hashing the literal text rather than a parsed value is the conservative choice: `60` becoming `60.0` is treated as a change, so the gate never has to decide whether two spellings are equal. The gate does not bump the version itself. A version is a name stamped on users' data, and requiring a person to choose the new name makes each algorithm change a deliberate, reviewable act. Sixteen hex

characters keep the baseline on one human-readable line while leaving accidental collisions out of practical reach. Exit code 2 for a missing input separates "cannot measure" from "measured a violation", so neither is ever reported as clean.

Ranking-weight confinement (same script). The probable-cause list orders candidate causes by a hand-set number named `rankingWeight`. Its register entry records that the value has no outcome evidence and must never be rendered as a confidence or probability. The gate walks every Swift file under the app sources. Any line containing the substring `rankingWeight` in a file outside the cause-detection directory is a violation.

Why it works. An ordering key is a legitimate internal tool, but the same number shown as "72 %" reads as a probability. Confining the identifier to one directory keeps it out of views, exports, analytics and accessibility text at the point of use. Moving it elsewhere requires a rename, and a rename is a visible change that a reviewer reads against the register entry.

Version stamp at runtime.

Rule	Behaviour
Construction	A new <code>ScoreBreakdown</code> defaults its <code>scoringVersion</code> field to the current version
Decoding	A stored breakdown with no <code>scoringVersion</code> key decodes to the literal string <code>unversioned</code> , never to the current version. Scores from before the stamp may come from v1 or v2, because the app offered a history recompute the user may or may not have run.
Derived copies	Code that rebuilds a breakdown copies the version field forward. This covers the vitals refresh after scoring and the stripped copy prepared for iCloud sync.
Display	The recovery-score detail screen shows "Scored by v3.1.oct2026" (the stored value), or "Scored before score versions were recorded" for <code>unversioned</code> . Beneath it is a fixed line saying the inputs are research-informed and the weights and bands are calibrated, not outcome-validated.
Assistant	The in-app assistant's fact catalogue (key <code>score.algorithm.version</code>) and its context block report the current version string and describe the v1 weighting older history may carry.

Why it works. Freezing and versioning each stored score keeps history attributable: a user comparing this month with last month can see whether the same algorithm produced both. The `unversioned` sentinel is the honest default for old records. Their true version is unknowable, because a recompute may or may not have run, and labelling them with the current version would assert something false. Carrying the stamp through every derived copy matters because a stamp dropped on sync or refresh would quietly re-label old scores as new. Placing the stamp next to the fixed "calibrated, not outcome-validated" line puts the version and the evidence status in front of the user together.

Version history in the public repository.

Version	Introduced
v2.may2026	2026-09-08 (public initial commit)
v3.oct2026	2026-10-03
v3.1.oct2026	2026-10-04

3.2 The science register

Data model. One JSON file with these top-level keys:

Key	Content
<code>_purpose</code>	Prose: a governed inventory of heuristics that change a number the user sees
<code>_what_the_gate_can_and_cannot_prove</code>	Prose statement of the gate's scope (quoted in Section 5)
<code>_retracted_claims.claims</code>	List of evidence phrasings that were once stated and later corrected (3 strings at this revision: two spellings of one reliability range, and one citation to a replication study)
<code>_status_meanings</code>	Map from each allowed status word to its definition. The gate reads the vocabulary from this map.
<code>_waived_symbols</code>	Map from function name to the reason it is not an entry (1 at this revision: a display-only translation helper)
<code>scoring_version</code>	Must equal the code's current version
<code>entries</code>	List of classified heuristics

Status vocabulary (exact words and meanings from the register).

Status	Meaning (register text, abridged)	Count
<code>validated</code>	An external peer-reviewed result supports this specific use, or the repository verifies it against reference data on every build	3
<code>supported-transfer</code>	A real published result exists, but the app applies it outside the population, protocol or endpoint it was established in. Defensible, not established.	4
<code>awaiting-validation</code>	A hand-calibrated product rule with no outcome evidence for these particular numbers. Retained because it is bounded and removing it would change stored scores; version-stamped so a later removal is visible.	12
<code>scheduled-for-removal</code>	Kept only until a replacement lands	0

Entries at this revision.

Status	Entries
<code>validated</code>	<code>hrv-time-domain-arithmetic</code> , <code>lnrmssd-swc-band</code> , <code>dfa-artifact-rejection</code>
<code>supported-transfer</code>	<code>dfa-a1-exercise-threshold</code> , <code>hrr-12bpm-band</code> , <code>training-load-ladder</code> , <code>tanaka-hrmax</code>
<code>awaiting-validation</code>	<code>resting-dfa-a1-reference-band</code> , <code>lfhf-window-filter</code> , <code>pns-sns-gap-readiness</code> , <code>acwr-readiness-damper</code> , <code>composite-weights-60-25-15</code> , <code>sleep-score-six-factor</code> , <code>hrv-sleep-staging</code> , <code>vitals-illness-pattern</code> , <code>spo2-flat-penalty</code> , <code>cause-ranking-weights</code> , <code>rmssd-absolute-fallback-bands</code> , <code>baseline-staleness-penalty</code>
<code>scheduled-for-removal</code>	none

Entry fields.

Field	Required by the gate	Role
<code>id</code>	Unique	Stable name
<code>status</code>	Must be a key of <code>_status_meanings</code>	Evidence level
<code>affects</code>	Non-empty	Which user-visible outputs change
<code>evidence</code>	Non-empty	What supports or does not support the rule, with sources
<code>allowed_language</code>	Non-empty	The wording copy may use, and the wording it must never use (for example "Damper, bookkeeping. Never injury risk or dangerous.")
<code>tests</code>	Optional; every path listed must exist	Tests that pin the behaviour
<code>symbols</code>	Optional	Function names in the analysis layer that this entry claims

Gate steps.

- Exit 2 if the register, the entry-floor file or the version source is missing, if the JSON does not parse, or if `_status_meanings` is empty.
- For each entry, record a problem for a duplicate `id`, a status outside the vocabulary, an empty `affects`, `evidence` or `allowed_language`, or a `tests` path that does not exist in the working tree.
- Read `current` from the version source and record a problem if the register's `scoring_version` differs. Every version bump therefore fails this gate until the register is edited.
- Retracted claims.** For each retracted string, search for an exact substring match in:
 - each entry's `evidence`, `affects`, `allowed_language` and `notes` fields (lists joined with spaces);
 - every `.swift` and `.md` file under the app sources and the docs directory.

The retracted list itself is the only place the strings may appear. This paper describes them without quoting them for that reason.
- Discovery.** Over every Swift file in the analysis directory, excluding the cause-detection subdirectory, collect the names matched by `func\s+([a-zA-Z]+(?:Adjustment|Penalty|Bonus|Modifier|Damper|Fraction))\b`. Discard names ending in `Descriptions` or `Message`. At this revision the scan finds 13 functions: 12 claimed and 1 waived.
- Let `claimed` be the union of all entries' `symbols`, and `waived` the non-underscore keys of `_waived_symbols`.
 - Each discovered name in neither set is a problem ("a score-adjusting function no register entry claims").
 - Each name in `claimed` $\cup$ `waived` that was not discovered is a problem ("claimed or waived but no longer exists"). A stale claim fails, just as a missing one does.

7. Print a tally by status, plus the discovered, claimed and waived counts. If there are any problems, exit 1.
8. **Entry floor.** Read the minimum entry count from the CI directory's register floor file. If the number of entries is below it, exit 1 with instructions: a heuristic still in the code must stay classified, and a heuristic removed from the code needs the floor lowered through the audited trailer of Section 3.5. If the count is above the floor, print a reminder to raise it.

The floor file's name begins with `min_`, so the ratchet of Section 3.5 treats it as a floor that may only rise.

Why it works.

- *A closed vocabulary read from the register's own definitions.* A status word is only useful if everyone reads it the same way. Keeping each word's definition beside the word, and deriving the allowed set from that map, means no status can be used without a written meaning.
- *A per-entry wording boundary.* What may honestly be said about a heuristic depends on its evidence, and differs from one heuristic to the next. A global banned-word list cannot express "may say 'damper', may never say 'injury risk'" for one rule and something else for another. `allowed_language` puts that boundary in the same record as the evidence that justifies it.
- *Version agreement.* The moment the algorithm changes is the moment its evidence statements may stop being true. Requiring the register's version to match the code's turns every algorithm change into a forced visit to the register.
- *A retracted-claims list.* A correction made in one file does nothing about copies of the old figure elsewhere. Keeping the corrected phrasings in one list, and failing the build wherever else they appear, gives each correction a single home and makes it permanent.
- *Name-shape discovery with a symmetric stale check.* The analysis layer names its score-adjusting functions by what they do (`...Penalty`, `...Damper`). A scan over that convention catches a new adjustment on the day it is written, at negligible cost. The reverse check, that every claimed name still exists, keeps the register from describing code that is gone.
- *An entry floor.* Deleting an entry would silently shrink what the register makes visible. A floor that only rises makes every removal an audited, deliberate act.

3.3 The copy perimeter, briefly

White Paper 3 covers the copy perimeter in detail. In outline, a Python linter reads a JSON list of prohibited regular expressions and scans every Swift string literal and string-catalogue value for user-facing medical-claim vocabulary. Comments are stripped, and an allowlist covers the methodology page and the guard sources. A second gate proves that the runtime vocabulary used by the in-app assistant's input and output guards covers every build-time pattern. For each build-time regex it synthesizes a matching string and requires some runtime lexicon concept to match it too.

For this paper, the point is the division of labour. The perimeter is lexical: it bans phrasings across the whole app. The register's `allowed_language` field states the claim boundary per heuristic, which is a judgement about meaning. The build forces the maintainer to revisit that boundary on every version bump (step 3 of 3.2), and the perimeter enforces the vocabulary that no boundary permits. Each mechanism does the part it can do reliably.

3.4 Gates that test the gates

The repository has 45 gate scripts (names beginning `check_` or `enforce_`). Three meta-checks establish three properties of every gate: it **goes red** on a real violation, it **fails closed** when it cannot measure, and it **is wired** into CI.

Why three properties. A green result means "nothing is wrong" only if the gate could have gone red, actually measured something, and actually ran. Each property fails independently: a gate with a typo in its pattern stays green on real violations; a gate whose temp file was not created greps nothing; a gate missing from the workflow never runs. Checking each property separately, for every gate, is what lets a green CI run be read as evidence.

3.4.1 Goes red: planted violations

For each plant, defined as (name, gate command, file to mutate, mutation program):

1. If a name filter was given and does not match, skip.
2. Run the gate. If it is already red, print SKIP ("fix that first") and continue. A red-after-mutation result proves nothing if the gate was red before.
3. Copy the target file to a backup directory, which was created through the fail-closed helper of 3.4.2.
4. Apply the mutation, a short Python program that edits the file. Most mutations assert that the original text occurs the expected number of times. If the mutation fails, print SKIP and restore.
5. Run the gate again, record its exit code, and restore every touched file.
6. A non-zero exit is PASS. A zero exit is FAIL ("STAYED GREEN — this gate protects nothing").

At this revision there are 64 plants. Examples: a retracted phrase appended to a register evidence field; an unknown register status; the hash baseline zeroed under the same version; a budget raised by 100000 with no trailer; a CI step renamed to a nonexistent script; the pNN50 mutation left in place; a bare `mktemp` in a gate. Several gates have more than one plant, one for each shape of miss the gate is meant to catch.

Meta-check: every gate has a plant. After the plants run, the harness lists every `check_*.sh` and `enforce_*.sh` in the scripts directory and drops two groups. The first is a two-entry `NEEDS_BUILD` exemption list (the coverage-floor gate and the Thread Sanitizer gate, which need a build or a simulator). The second is the preflight self-check, whose name is excluded from the meta-check although it does have two plants. For each remaining gate it greps the harness's own source for the gate's filename. Any gate whose filename does not appear makes the run exit 1, listing the unplanted gates. Adding a gate and adding its plant must therefore happen in the same change.

The run also exits 1 if any plant was FAIL. A plant that was SKIPped is reported and does not fail this harness; the test-mutation harness of 3.4.4 treats skips more strictly.

Why it works. The only convincing evidence that a check can detect a violation is a violation it detected. Requiring green first rules out a gate that is red for an unrelated reason. The completeness meta-check is what makes the discipline scale: the set of gates that must have plants is derived from the directory, not from anyone's memory, so coverage grows with every new gate automatically.

3.4.2 Fails closed: temp-file preflight

The gates run with `set -uo pipefail` but not `-e`, because several rely on `grep` returning non-zero when nothing matches. Without `-e`, a failed `mktemp` leaves an empty path, and later reads of that path find nothing, which a gate would report as "clean". The design has two halves.

Helper library.

- `gate_mktemp label` creates `${TMPDIR:-/tmp}/label.XXXXXX` with an explicit template, which behaves the same under BSD and GNU `mktemp`. If creation fails, the path is empty, or the file is not writable, it prints the reason and calls `exit 70 (EX_OSERR)`.
- `gate_mktemp_dir` does the same for directories.
- `gate_require_nonempty` exits 70 when an input the gate is about to parse is missing or empty.

Each helper runs inside `$( )`, which is a subshell, so its `exit` cannot stop the caller. Every call site must therefore append `|| exit $?`.

Preflight gate. For every `.sh` file in the scripts directory except itself, it scans each non-comment line and reports:

1. any `mktemp` not preceded by `gate_` on a line that does not contain `gate_mktemp`;
2. any line calling `gate_mktemp` without `|| exit` on the same line;
3. any `source "$(dirname "$0")/<path>"` whose target does not exist, because a mistyped source path would silently define no helpers.

Any finding exits 1. At this revision it scans 53 scripts and finds 10 guarded temp-file uses.

The ratchet comparison (3.5) and the scoring and register gates also fail closed on configuration problems: a missing input exits 2, not 0.

Why it works. Fail-open behaviour comes from one predictable place, an unchecked resource whose absence looks like an empty result. Routing every temp-file creation through one helper puts the failure handling in a single audited function, and the static scan makes that routing mandatory. Checking the `|| exit $?` guard at the call site closes the subshell gap that would otherwise let the helper's exit vanish.

3.4.3 Is wired: CI reachability

The wiring gate reads the manual full-CI workflow file. The path can be overridden by an environment variable; a missing file exits 2. Every `check_*.sh` or `enforce_*.sh` whose filename does not occur as a substring of that file is reported. One gate is exempt by name with a recorded reason: the coverage-floor gate, which runs inside the coverage step. Any unwired gate exits 1. At this revision 44 gates appear in the workflow and 1 is invoked by another step.

Why it works. A gate that exists but never runs protects nothing and looks complete. Deriving the list of gates from the directory and the list of steps from the workflow makes reachability a property the build checks, rather than something a reviewer infers.

3.4.4 The tests must catch defects: targeted mutations

This harness applies the planted-violation discipline to the unit tests instead of the gates. It holds a curated list of 125 mutations, each defined as (name, test suite identifier, production file, mutation program). Examples: widen the pNN50 threshold from 50 ms to 75 ms; return the mean instead of the standard deviation for SDNN; feed raw RR intervals instead of successive differences to RMSSD; shift the top verdict band from 90–100 to 91–100; remove the finiteness guard on a date-to-milliseconds conversion; drop the detrend step from DFA.

For each mutation:

1. Run the named suite alone (`-only-testing`, parallel testing off) and classify the result:
 - **0** — passed;
 - **1** — tests failed;
 - **2** — build or runner failure. This is recognised by a build-failure banner in the log, or by the absence of any "Test Suite" or "Test Case" line.
2. If the baseline result is 2, retry once. If the baseline is still not 0, print SKIP with the reason.
3. Back up the file and apply the mutation. Each mutation asserts that its original text occurs exactly once, so a mutation cannot silently land in a doc comment or the wrong place. If it cannot be applied, print SKIP.
4. Run the suite and restore.
5. A non-zero result is PASS ("caught"). A zero result is FAIL ("SURVIVED — asserts nothing about this").
6. At the end, **any FAIL or any SKIP exits 1**. "A skip is not a pass."

This harness needs a simulator and a build per mutation. It is not part of `make ci`, and no CI workflow runs it; it is run by hand when analysis or storage code changes.

Why it works. For a physiology score, the important question is not what fraction of random mutants die but whether a specific wrong formula would be caught. Each curated mutation is a plausible scientific error (the wrong threshold, the wrong statistic, the missing detrend), and naming its suite turns a survivor into a precise finding: this suite does not assert this behaviour. Running the suite alone keeps attribution exact. Classifying build failures separately stops a broken build from counting as a catch, and treating every skip as a failure stops an unapplied mutation from passing silently.

3.4.5 Backstops: leaked-mutation detection and the tree lock

Both harnesses edit tracked files in place and restore them in an EXIT trap. A killed process skips the trap, so two backstops cover that case.

Leak detector. A static gate parses the mutation harness's source:

1. Split the source on the literal line prefix `mutate "`.
2. For each case, take the target path: the first double-quoted string that starts with the app directory and ends in `.swift`. Take the original text from either `o='...';assert` or `s.count('...')` followed by a comparison operator.
3. Unescape `\'`, `\"`, `\n` and `\$`. Resolve `' + chr(N) + '` concatenations to the character `N`.
4. If the original still contains `chr(` or a string concatenation, mark the case unparsed.
5. If the target file is missing, or the original text is not present in it, report a leak.

6. Any leak exits 1. **Any unparsed case also exits 1**, because a case the parser cannot read is a case it is not checking.

This gate runs on every push.

Tree lock. Before mutating, each harness atomically creates a lock directory inside the git directory using `mkdir`, and writes its name and PID into it. A second mutator finds the lock; if the PID is alive, it exits 75 (`EX_TEMPFAIL`). If the PID is dead, it warns that the earlier run never restored, points to the leak detector, and takes the lock over. Read-only checks can call a reader function that refuses to run (exit 75) while a live mutator holds the lock. The lock is released in the same trap that restores files.

Why it works. The mutation list already states, for every case, the exact text that must be present in the clean tree. Reading that list statically turns it into a manifest that exists before any run, so no run has to record one, and a leak from a run killed at any point is still found. Because the detector needs only Python and the repository, it runs on every push on Linux, far more often than the harness itself. Creating a directory with `mkdir` is atomic, and keeping it inside the git directory keeps it out of the working tree and out of commits.

3.5 One-way ratchets

Decision record. The repository's architecture decision record on ratcheted budgets (ADR 004) states the policy. Every quality rule has a measured numeric ceiling, never an aspirational one. A ceiling may only move down. There is no per-site escape hatch: the app sources contain no SwiftLint disable directives. The record reasons that a gate nobody can satisfy gets routed around, while a measured ceiling blocks regression immediately.

Counting code, not prose. Several budgets count occurrences of a pattern: static singleton reads, `try?`, legacy observable-object conformances. Before counting, the counter removes comments. Block comments `/* ... */` are blanked character for character, keeping newlines. Line comments `//...` are replaced with spaces of the same length, so nothing on an adjacent line is joined into a match. Explaining a piece of debt in a comment therefore never raises its count. A fixed list of platform singletons (`URLSession`, `FileManager`, `NotificationCenter` and others) is blanked before the singleton count, because no version of the app can inject those away.

Monotonicity gate.

1. **Baseline resolution**, in order:
 - an explicit argument;
 - the pull-request target branch (`origin/<GITHUB_BASE_REF>`);
 - a `BUDGET_BASE_REF` environment variable;
 - otherwise there is no baseline.
2. **No baseline.** If `BUDGET_REQUIRE_BASELINE` is set, exit 1 ("refusing to report enforcement that did not happen"). Otherwise print a skip notice and exit 0. The same applies to an all-zero SHA (a new or force-pushed branch) and to a ref that does not resolve.
3. **Acknowledgements.** Collect them from every commit message in `baseline...HEAD`, using the case-insensitive regex `^budget-raise-ok:[[:space:]]*[^[:space:]]+`. Keep only the path token.
4. **Comparison.** For each `*.txt` file directly in the CI directory, in sorted order:
 - Read the current value. It must match `^[0-9]+$` once whitespace is removed; otherwise skip the file.

- Read the value at the baseline with `git show`. A file absent at the baseline is a new budget and is skipped. A non-integer baseline value is skipped too.
- Equal values pass.
- **Direction** comes from the filename. A basename starting with `min_` is a floor, and a decrease loosens it. Any other file is a ceiling, and an increase loosens it.
- A loosening whose exact path appears among the acknowledged paths prints `ALLOWED`. Any other loosening is a violation. A tightening prints `ratcheted`.

5. Any violation exits 1, with instructions to pay the debt or add `budget-raise-ok: <path> <why>`.

CI wiring of the baseline. The manual workflow resolves the baseline to the parent commit, or to the working tree for a root commit with budget edits, and sets the `require` flag whenever a baseline exists. An unresolvable baseline is a hard failure. The per-push workflow passes the pull-request target, or the parent commit with `BUDGET_REQUIRE_BASELINE=1` on a direct push to main. A push to main therefore cannot skip the comparison.

Documentation may not restate a ratcheting value. The documentation-links gate scans every Markdown file. Whenever a line names a CI budget file whose content is a plain integer, it looks for a bare 2–6-digit number in that line or the next one, using the regex `(?<![\w. ])(\d{2,6})(?![\w.%])`. Any such number is reported. The one exception is when the window contains a backtick code span with `cat`, `head` or `read_budget`, which is a live read and cannot go stale. Docs must point at the file instead of quoting it. This paper follows that rule, and gives no ratcheting budget value next to its file name.

Why it works.

- *Measured, not aspirational, limits.* A ceiling set at today's measured value is satisfiable immediately, so nobody is tempted to route around it, and it blocks the next regression on the day it would land.
- *Direction in the filename.* The rule that governs a budget travels with the budget. A new budget declares its direction when it is created, there is no separate configuration that could drift from the files, and a reader can tell from a directory listing which numbers may rise and which may fall.
- *An audited escape, not a silent one.* Some loosening is legitimate, for example when a heuristic is deliberately removed. The trailer allows them, but only in the commit history, attached to the named file, where a reviewer and every later reader can see it.
- *A required baseline in CI.* A comparison that did not happen must not look like one that passed. Locally, the gate skips without a baseline so that uncommitted work needs no justification addressed to no one; in CI the `require` flag makes the same condition a failure.
- *No restated values in docs.* A number copied into prose is a second copy that the ratchet cannot update. Pointing at the file keeps one source of truth.

Observed history. Across the 28 commits of the public repository (2026-09-08 to 2026-10-07), every integer budget that changed moved in its tightening direction. The SwiftLint warning, `try?`, test-skip and localization-orphan ceilings each fell more than once, and the refactor-spec watch ceiling fell to zero. The views-coverage floor rose. No commit message carries a `budget-raise-ok: trailer`.

3.6 Costed CI posture and the evidence report

macOS CI minutes bill at ten times the Linux rate. The repository measured a full macOS run at about 108 wall-clock minutes, roughly 1,080 billable, against a plan of about 300 macOS minutes a month. The posture follows

from that arithmetic:

Workflow	Trigger	Runner	Content
Script gates	Every push to main and every pull request	Linux	Every gate that needs only bash, Python and git: copy perimeter, perimeter sync, science register, scoring governance, localization, source-hygiene guards (including the leaked-mutation backstop), documentation and SBOM checks, wiring and preflight meta-gates, tech-debt budgets, and the monotonicity ratchet
Full CI	Manual, or called by the release workflow	macOS	Scope <code>gates</code> (about 14 billable minutes): SwiftLint and its budget, all budgets, the planted-violation harness and every gate step. Scope <code>unit</code> adds unit tests and the Thread Sanitizer. Scope <code>full</code> adds UI tests and coverage floors.
Local	<code>make ci</code>	Developer machine	Every gate, the planted-violation harness and the full test suite with coverage
Manual	By hand	Developer machine with simulator	Targeted mutation harness (3.4.4)

The planted-violation harness runs in the manual workflow and in `make ci`, where SwiftLint behaves as it does for the app; the per-push Linux workflow carries every gate that does not depend on it.

The **evidence report** script prints, in seconds and without building, live counts (files, tests, locales, waivers) and every budget value, as text or JSON. It omits coverage and pass/fail results on purpose: those need a simulator, and a cached number would be stale.

Why it works. Everything that governs scientific honesty (the hash lock, the register, the perimeter, the meta-gates, the ratchet and the leak backstop) needs only bash, Python and git. Running all of it on every push on Linux costs almost nothing, so the guarantees that matter most are checked most often, and the expensive simulator work is spent where only a simulator can help.

3.7 How the parts form one chain

1. A developer edits a numeric threshold in a scoring-constant file. **Scoring governance** fails: the digest moved and the version did not.
2. The developer changes `ScoringVersion.current` and re-records the baseline. New breakdowns carry the new stamp; stored ones keep theirs.
3. **Science register** fails until its `scoring_version` is edited, which is the moment to revise each affected entry's status, evidence and `allowed_language`.
4. A new score-adjusting function fails discovery until claimed or waived; a removed one fails the stale-claim check; a removed heuristic fails the entry floor unless the floor is lowered with an audited trailer.
5. New user-facing copy passes through the **copy perimeter** (White Paper 3), and every gate involved is proven to go red, fail closed and be reached by CI.

Why it works. Each gate is small and cheap. Their value is the coupling: the one event that changes what a score means, a change to its numbers, triggers in order a new name for the algorithm, a stamp on the user's data, and a

forced review of what may be said about it.

4. What Is New

Each statement below was framed after a prior-art search made on 2026-10-08. The search covered PyPI, npm, NuGet and GitHub, vendor and tool documentation, and the literature cited in the references. "Builds on" names the closest published work and what the statement adds to it. "Earliest dated record" gives the commit in the author's private repository lineage (Section 8) where the method first appears, and the commit where it reached its current form when that differs. The private repositories are not public, so these dates show when the author built each method; the public disclosure date for every statement below is 2026-09-08, the first commit of the public repository. Statement numbering follows v1.1.

N1. A build gate that hashes only the comment-stripped numeric `static let` declarations of designated scoring-constant files, as a per-file sorted list of `path:symbol=value` lines under SHA-256 truncated to 16 hex characters, and fails the build when that digest differs from the recorded digest while the scoring-version string, the same string stamped on every stored score (N2), still equals the version recorded beside it. Builds on: hashing configuration or lockfiles to detect change, which is general practice. CI checks that fail when hashed content changes without a semantic-version bump were later proposed in maiconsouza89/agents-skills issue 130 (2026-09-17) [27] and pandas-studio/agent-team-plugins pull request 165 (2026-09-27) [28], both after Emuqu's public commit. Adds: selection of numeric declarations only, comment stripping, and a tie to the version stamped on stored user data. Earliest dated record: flow-recovery dba6fcc, 2026-05-01, as a precursor: a unit test that pinned the version string of a versioned scoring-parameters value and its six score-band values, failing if a value changed without a version change. Hash gate in its current form: emuqu-dev f2e6ed1, 2026-09-03.

N2. Persisting the scoring-version string on every stored score breakdown, decoding records written before the field existed to an explicit `unversioned` sentinel rather than to the current version, carrying the stamp through every derived copy, and rendering the stored stamp (or a "scored before versions were recorded" line) on the score-detail screen next to a fixed statement that the weights are calibrated, not outcome-validated. Builds on: version identifiers on machine-learning models in model cards and registries [7][8]. Adds: a version stamped on each stored consumer score and shown to the user. No wearable vendor documentation found describes score versioning (searched: WHOOP, Oura, Garmin and Polar public documentation). Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03 (precursor: the versioned scoring parameters of flow-recovery dba6fcc, 2026-05-01, which were not persisted with scores).

N3. A containment gate that confines a hand-set ranking-weight identifier to a single source directory, so that a number with no outcome evidence cannot be displayed as a confidence anywhere else in the app without a visible rename. Builds on: general lint rules restricting identifiers. Adds: confinement tied to an evidence-register entry. No prior instance found in a search of PyPI, npm, GitHub and lint-rule documentation. Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03.

N4. A register of the score-adjusting heuristics in an app, in which each entry must carry a non-empty `allowed_language` field stating the user-facing wording it permits and forbids, and in which the register's declared scoring version must equal the code's current scoring version, so that every scoring-version change fails the build until the register is edited. Builds on: sphinx-needs, which restricts item statuses to a configured list of

names with descriptions and enforces required fields by schema [22]; Doorstop [24] and reqcov [23] (PyPI, 2026-09-04), which check that requirements are linked to tests and code. A closed status vocabulary, required fields and existence checks on cited tests are therefore not claimed. Adds: a tie from the inventory to a code version, and a per-item wording boundary. Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03.

N5. A retracted-claims list kept inside the register whose strings fail the build if they reappear in any register entry or in any Swift or Markdown file under the app sources and docs, making the register the only place a corrected error may be written. Builds on: requirements tools that keep an inventory in the repository (sphinx-needs, Doorstop, reqcov). Adds: a list of corrected phrasings enforced across the code and documentation. No prior instance found in a search of PyPI, npm, GitHub and requirements-tool documentation. Earliest dated record: emuqu 9d89933 (public first commit), 2026-09-08; it is absent from emuqu-dev.

N6. Name-shape discovery of score-adjusting functions (names ending in Adjustment, Penalty, Bonus, Modifier, Damper or Fraction) in the analysis layer, in which every discovered function must be claimed by a register entry's `symbols` list or waived by name with a reason, every claimed or waived name must still exist, and the register's entry count is a floor that only rises. Builds on: traceability tools that check links from requirements to code [22][23][24]. Adds: discovery of unregistered code by name shape, with a symmetric stale-claim check and a rising entry floor. No prior instance found. Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03 (discovery and stale-claim check). Entry-count floor in its current form: emuqu 9d89933, 2026-09-08.

N7. A planted-violation harness that refuses to report success while any `check_/enforce_` script in the scripts directory lacks a plant. It searches its own source for each gate's filename and fails the run if one is absent, so that adding a gate without a plant fails the build. A named list exempts only the gates that need a build or simulator. Builds on: canfail (PyPI, 2026-08-31) [19], which breaks a file on purpose, requires the check to go red, confirms the check is green first, requires the anchor to match exactly once and verifies the restore. Those rules are therefore not claimed. Adds: completeness, a requirement that every check in the repository has a plant. Earliest dated record: emuqu-dev 26fe7b1, 2026-08-28 (plants, green-first). Completeness check: emuqu-dev ba968cd, 2026-09-02.

N8. A preflight gate that statically requires every gate's temp-file creation to go through a helper that exits non-zero on failure, with an `|| exit $?` guard at every command-substitution call site, and requires every sourced helper path to resolve, so that gates fail closed instead of reporting clean after measuring nothing. Builds on: didrun (PyPI, 2026-08-31) [20], which requires positive evidence in a check's output that it ran. Adds: a static check of the gate's own source for the temp-file failure class. Earliest dated record: emuqu-dev 26fe7b1, 2026-08-28.

N9. A wiring meta-gate that fails when any `check_/enforce_` script's filename does not appear in the CI workflow file, with a named exemption for a gate invoked by another step. Together with N7 and N8 it makes "is wired into CI" a third separately enforced property of every gate, next to "goes red" and "fails closed", and each of the three meta-gates is itself covered by a planted violation. Builds on: canfail [19] (goes red), didrun [20] (ran) and Semgrep's `ruleid:` test annotations [29], which prove that a rule fires on an example. Adds: a check that every gate is reached by CI. No prior instance found. Earliest dated record: emuqu-dev ba968cd, 2026-09-02.

N10. A curated production-mutation harness in which each mutation names the single test suite that must catch it and that suite is run alone; a baseline run classified as a build or runner failure (a build-failure banner, or no "Test Suite"/"Test Case" line in the log) is retried exactly once; and any skipped mutation (baseline not green, or mutation not applicable) fails the whole run. Builds on: mutation testing in general [9][10][13][14], which is not

claimed, and canfail [19], which also requires a green baseline and an anchor that matches exactly once, and scores a syntax error, or a run with no evidence it ran, as not a catch. Those rules are not claimed. Adds: per-mutation suite attribution, a single classified retry, and skips that fail the run (in canfail such cases are reported without failing it). Earliest dated record: emuqu-dev d0267b3 "Add mutation testing", 2026-08-28 (anchor exactly once, green first). Build/test classification, single retry and skip-fails: emuqu-dev 409a762, 2026-09-01.

N11. A static leak detector that parses the mutation harness's own source to recover each mutation's target file and original text (including `chr()`-escaped forms) and fails if that text is absent from its target file or if any case cannot be parsed, combined with an atomic `mkdir` lock in the git directory, shared by both mutation harnesses, that records the holder's PID, refuses a second mutator while that PID is alive, takes over a dead holder's lock with a warning, and makes read-only checks refuse to run while a live mutator holds it. Builds on: restore-verified (PyPI, 2026-08-31) [21], which verifies a restore byte for byte against a digest recorded beforehand, with an outer sentinel that detects a tree left modified after SIGKILL. Adds: detection that needs no manifest recorded before the run, and locking between mutators and readers. Earliest dated record: emuqu-dev ba968cd, 2026-09-02.

N12. A budget ratchet over a directory of single-integer files in which the allowed direction is read from the filename (`min_` prefix: a floor that may only rise; any other name: a ceiling that may only fall), a loosening passes only when a commit in the compared range carries a `budget-raise-ok: <path>` trailer naming that file, and a CI-set flag turns a missing baseline into a failure. Builds on: jest-ratchet (npm, 2018) [26] and CoverageRatchet (NuGet, versions listed from 2026-05-05) [25], which ratchet coverage thresholds upward, and betterer [6] and SwiftLint baselines, which freeze existing results. Ratcheting as an idea is not claimed. Adds: per-file direction from the filename, an audited per-file trailer, and a required baseline in CI. Earliest dated record: emuqu-dev 17f1114, 2026-08-22 (`min_` floors and trailer). Require-baseline flag: emuqu-dev 6a7ff33, 2026-08-31.

N13. A documentation gate that rejects any Markdown line naming a ratcheting budget file next to a bare number, so prose must point at the value instead of restating it. Builds on: documentation link checkers in general. Adds: a rule tying prose to ratcheting values. No prior instance found in a search of PyPI, npm, GitHub and documentation-lint tools. Earliest dated record: emuqu-dev 8c3999d, 2026-08-26.

N14. Composing N1, N4, N6 and a lexical copy perimeter into one chain in which a change to a scoring constant forces a version change, the version change forces an edit of the evidence register that states each heuristic's permitted wording, and that wording is policed for prohibited vocabulary by the copy perimeter. Builds on: the individual tool families above. Adds: the composition. No prior instance found. Earliest dated record: emuqu-dev f2e6ed1, 2026-09-03.

Prior art not claimed.

- Recording a baseline of existing lint findings and failing only on new ones: SwiftLint's baseline option and betterer [6]. Coverage thresholds that only rise [25][26].
- Mutation testing and mutation scores in general [9][10][13][14].
- Breaking a file on purpose to prove a check goes red, with a green-first baseline, an anchor that must match exactly once, and a verified restore [19]; requiring evidence that a check ran [20]; verifying a restore byte for byte and catching a tree left modified after the mutating process is killed [21].
- A closed status vocabulary with descriptions, required fields per item, and checks that requirements link to existing tests and code [22][23][24].
- Version identifiers and documentation cards for models [7][8], and semantic versioning of software [15].

- Cryptographic hashing of configuration or lockfiles to detect change, and CI checks that require a version bump when hashed content changes [27][28].
- Git commit trailers as a convention.
- Traceability between requirements, risk controls and verification as required by medical-device standards [11][12][16].
- The HRV methods named in the register, which are cited to their published sources [1][2].
- The general observation that ML and data systems accumulate configuration and documentation debt [17][18].

5. Design Tradeoffs

Recorded baseline, not history. The hash lock compares the code with one recorded line, which keeps the gate stateless and readable; the recorded line itself is trusted, so edits to it are left to review.

Two files, literal declarations. The lock covers the numeric `static let` literals of two designated files; the composite weights live in a separate weights namespace in the general constants file and are governed through their register entry (`composite-weights-60-25-15`) rather than the hash, and inline literals and expression-valued constants are outside it.

Truncated digest. Sixteen hex characters detect accidental change; they are not meant to resist deliberate collision.

Lexical checks. Ranking-weight confinement, name-shape discovery and the plant and wiring checks match names and filenames, which keeps them fast and deterministic at the cost of seeing only the shapes they look for; the register states this itself: "governance over a known failure class, not proof that nothing was omitted."

Structure, not truth. The register gate checks fields, links, versions and phrasings; whether a status is honest, and whether copy matches an entry's `allowed_language`, remain maintainer judgements that the version gate forces to be revisited.

Targeted, manual mutation. Curated mutations name the behaviour at stake but cover only the behaviours listed, and the harness runs by hand because each case needs a simulator build; the static leak detector is its standing check on every push.

Measured ceilings. Setting each ceiling at the measured value tolerates existing debt, which ADR 004 calls "debt inventories rather than assurance targets", in exchange for gates that can be satisfied on the day they land.

Scope. None of this verifies that the science is right; it establishes that numbers, versions, statuses, citations and wording stay consistent with one another and cannot change silently.

6. Verification

The governance is self-verifying in the sense of Section 3.4, and its counts at e028039 are printed by the gates themselves:

Check	Result at e028039
Constants hashed by the scoring-governance gate	155 (91 + 64)
Score-adjusting functions found by discovery	13: 12 claimed, 1 waived
Planted violations in the gate harness	64, with the every-gate-has-a-plant meta-check applied to all gates outside the two-entry build exemption
Scripts scanned by the preflight gate	53, with 10 guarded temp-file uses
Gates reached by the full-CI workflow	44 named in the workflow, 1 invoked by the coverage step
Targeted test mutations parsed by the leak detector	125 of 125 parsed; no leaked mutation
Integer budgets changed across the 28 public commits	Every change in its tightening direction; no <code>budget-raise-ok</code> : trailer used

The targeted-mutation harness itself is described here as a method; a run at e028039 is not reported in this paper.

The analysis methods that the register classifies were independently re-implemented in Python from the Swift source at e028039 and run on PhysioNet recordings on 2026-10-08. Each port reproduced the app's own unit-test expectations, 114 cases across eight test suites. On the 20 nsr2db fixture records (6,000 intervals), mean RR and SDNN matched the stored references to within 4.9×10^{-7} ms, consistent with the `validated` status of the `hrv-time-domain-arithmetic` entry. The full independent validation report is `Tools/validation/RESULTS.md` [30].

7. Comparison to Existing Approaches

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
SwiftLint baseline	New lint violations fail; recorded ones are tolerated	Not numeric; the baseline can be regenerated	No	No	No	SwiftLint documentation
betterer	A test's result may not get worse than the stored result; improvements are written back	Per-test "better" comparator	No	No	No	[6]
jest-ratchet (npm, 2018)	Coverage thresholds rewritten upward as coverage improves	Coverage only rises; fixed by the tool	No	No	No	[26]

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
CoverageRatchet (NuGet)	Per-file coverage thresholds ratcheted automatically from Cobertura XML	Coverage only rises; fixed by the tool	No	No	No	[25]
Hash-without-version-bump CI checks (2026-09-17, 2026-09-27)	CI fails when hashed content changes and the semantic version does not; both post-date Emuqu's public commit	Not applicable	Yes, for the hashed files	No	No	[27][28]
Model cards	Structured documentation of intended use, evaluation and caveats	Not applicable	Documents a model version; not build-enforced	Yes, as prose	No	[7]
ML model registries (e.g. MLflow)	Versioned model artifacts with stage transitions	Not applicable	Versions artifacts; does not watch source constants	Metadata, free-form	No	[8]
sphinx-needs	Requirement items in documentation; status restricted to a configured list of names with descriptions; required fields by schema	Not applicable	No	Yes: typed items, statuses, links	No	[22]
Doorstop	Requirements as version-controlled text files linked to each other and to code and tests	Not applicable	No	Yes: requirement items and links	No	[24]
reqcov (PyPI, 2026-09-04)	Requirements traced to tests and code; CI fails on uncovered requirements; traceability matrix	Not applicable	No	Yes: requirements with coverage status	No	[23]

Approach	What it enforces	Direction of limits	Ties numbers to a version	Evidence or claim inventory	Proves its own checks fail	Source
PIT / Stryker mutation testing	Mutation score over broad automatic mutations; Stryker can fail the build below a configured threshold	Threshold set by the user	No	No	Tests only, by percentage	[9][10]
canfail (PyPI, 2026-08-31)	Declared breaks must turn a check red; green baseline first; anchor exactly once; syntax-only failures and runs without evidence are not catches; restore verified	Not applicable	No	No	Yes, for each declared break; no requirement that every check has one	[19]
didrun (PyPI, 2026-08-31)	A check must show positive evidence in its output that it ran (a count, a pattern, a written file)	Not applicable	No	No	Ran-or-not, per run	[20]
restore-verified (PyPI, 2026-08-31)	A mutated file is restored and the restore checked byte for byte; an outer sentinel detects a tree left modified after SIGKILL; no locking	Not applicable	No	No	Covers the restore step of a mutation harness	[21]
IEC 62304 / ISO 13485	Documented life-cycle, change control, traceability, audited	Process, not numeric	Through configuration management, as a process	Through design and risk files, as a process	Through verification records, as a process	[11][12][16]
Emuqu (this paper)	Constants-hash/version lock; claim register with closed vocabulary, test-existence, version match, retracted claims and discovery; per-file direction ratchets	Ceilings fall and <code>min_</code> floors rise, checked against a baseline commit	Yes, for numeric constants in two files, and the version is stored on each score	Yes, machine-checked fields with a per-entry wording boundary, versioned with the code	Yes: a plant required for every gate, preflight, wiring check, 125 targeted test mutations, static leak detector and tree lock	This paper

Emuqu is a consumer wellness app. It is not developed under IEC 62304 or ISO 13485 and claims no conformity with them. They appear in this table only because traceability and controlled change are what those standards address by process, and the mechanisms here address a narrow part of the same concern by build-time checks.

8. Provenance and Dates

8.1 Project origin

The project began as a general fitness app. The author's GitHub repository `chrissharp80/AI-Fitness-Coach` has 19 commits from 2025-02-16 to 2025-02-17. They add Apple HealthKit heart-rate and HRV display and Polar H10 real-time heart-rate streaming over Bluetooth. They contain none of the methods in this paper. The earliest recovered HRV-specific design exchange is a ChatGPT user message from the author on 2025-02-22 at 15:44:18 UTC, proposing an app that measures HRV from a chest strap worn overnight just before the user wakes [31]. The same day's conversation lists "HRV trends" in a broader fitness-app feature set, and messages on 2025-03-04 and 2025-03-05 set out a build, deploy and test workflow and a validation plan for a VO_2 -max estimate. These records establish where the project came from. They do not date any method in this paper.

8.2 Repository lineage

Repository	Product name	First commit	Visibility
<code>chrissharp80/flow-recovery-old</code>	Flow HRV	2026-01-12	Private
<code>chrissharp80/flow-recovery-dev</code>	Flow Recovery	2026-03-01	Private
<code>chrissharp80/flow-recovery</code>	Flow Recovery	2026-03-15	Private
<code>chrissharp80/emuqu-dev</code>	Emuqu	2026-08-16	Private
<code>github.com/chrissharp80/emuqu</code> (PolyForm Strict 1.0.0)	Emuqu	2026-09-08	Public

Each repository's first commit carries over code from its predecessor. The author retains all five with full history.

8.3 Earliest commit for each statement in Section 4

N	First form	Current form
N1	<code>flow-recovery dba6fcc</code> , 2026-05-01 (precursor: unit test pinning a scoring-parameters version and its band values)	<code>emuqu-dev f2e6ed1</code> , 2026-09-03
N2	<code>emuqu-dev f2e6ed1</code> , 2026-09-03	same
N3	<code>emuqu-dev f2e6ed1</code> , 2026-09-03	same
N4	<code>emuqu-dev f2e6ed1</code> , 2026-09-03	same
N5	<code>emuqu 9d89933</code> , 2026-09-08	same
N6	<code>emuqu-dev f2e6ed1</code> , 2026-09-03	<code>emuqu 9d89933</code> , 2026-09-08 (entry floor)

N	First form	Current form
N7	emuqu-dev 26fe7b1, 2026-08-28	emuqu-dev ba968cd, 2026-09-02 (every gate must have a plant)
N8	emuqu-dev 26fe7b1, 2026-08-28	same
N9	emuqu-dev ba968cd, 2026-09-02	same
N10	emuqu-dev d0267b3, 2026-08-28	emuqu-dev 409a762, 2026-09-01 (build/test classification, single retry, skip fails)
N11	emuqu-dev ba968cd, 2026-09-02	same
N12	emuqu-dev 17f1114, 2026-08-22	emuqu-dev 6a7ff33, 2026-08-31 (require-baseline flag)
N13	emuqu-dev 8c3999d, 2026-08-26	same
N14	emuqu-dev f2e6ed1, 2026-09-03	same

canfail, didrun and restore-verified were published on PyPI on 2026-08-31 [19][20][21]. That is three days after the private first forms of N7, N8 and N10 (2026-08-28) and eight days before Emuqu's public commit (2026-09-08). The private dates show the methods were developed independently. They do not make the author's work the earlier disclosure, because the packages were public first. Section 4 therefore claims only what those packages do not contain.

8.4 Other dated records

These are the author's own records, held in the author's Google Drive.

- 2026-01-08, about 9 p.m. Central (Drive file created 2026-01-09 03:06:59 UTC): the pre-implementation design document "Design: Connection + Offline RR Collection Harness (iOS + watchOS) – v9.2 FINAL", saved from an AI design conversation four days before the first code commit. Its "Critical (Block Ship)" acceptance tests — a 0.25 Hz sine must land in the HF band, total power must match a known amplitude, RMSSD must be within 5 % of a reference implementation — are the earliest recorded instance of the rule that a number may ship only after a check against a known answer, which the gates in this paper later enforce in CI. Versions 1–8 have not been recovered.
- App-generated session reports named "Flow_Recovery_.pdf", dated 2026-02-16, 2026-02-18 and 2026-03-04, and a "Flow Recovery - Bluetooth" screen recording dated 2026-03-23.
- Project documents uploaded 2026-05-10 (README, audit and refactor notes dated 2026-05-06): they describe "Frozen Historical Scores", the copy linter's prohibited-terms list and CI budget files for tech debt, SwiftLint and large files. They show that budget files and a copy perimeter existed by May 2026. They do not describe the direction-from-filename rule, the trailer or any other statement in Section 4.

No non-git record dates any other method in this paper.

8.5 Public disclosures

Date	Disclosure	Relevance to this paper
From 2026-01-27	TestFlight beta of the app (45 testers by August 2026)	Distributes app builds to testers; it does not publish the source or the build scripts
2026-05-19	Substack article "A Powerfully Lazy Man's Way to Better Health"	Described dual internal and streamed RR capture merged in the morning, α 1-based "organized windows" and a search band of 30–70 % of sleep (White Paper 1). None of this paper's methods
2026-08-10	Substack article "How I Build Production Software by Directing AI"	Describes the author's way of directing AI to build the app (45 beta testers, over 6,500 sessions). The author's records do not show it describing any method in Section 4
2026-09-08	Public repository github.com/chrissharp80/emuqu	First public disclosure of every method in Section 4
2026-10-08	These white papers (v1.0, v1.1 and this v1.2)	Full written description

The author also shared the work privately with two colleagues, in January and March 2026. Those exchanges concerned the overnight measurement, not the methods in this paper.

8.6 Authorship

Chris Sharp is the sole author and committer. He wrote no code by hand: AI coding assistants wrote it under his direction, and some commits are attributed to "Claude"; he supplied the requirements, design direction, review, testing and acceptance [31].

Public scoring-version history.

- v2.may2026: present at the public initial commit, 2026-09-08.
- v3.oct2026: 2026-10-03.
- v3.1.oct2026: 2026-10-04.

This paper: v1.0, v1.1 and v1.2 published 2026-10-08. It describes source revision e028039 (2026-10-07).

References

1. Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology (1996). Heart rate variability: standards of measurement, physiological interpretation and clinical use. *Circulation* 93(5):1043–1065.
2. Plews DJ, Laursen PB, Stanley J, Kilding AE, Buchheit M (2013). Training adaptation and heart rate variability in elite endurance athletes: opening the door to effective monitoring. *Sports Medicine* 43(9):773–781.

3. Peake JM, Kerr G, Sullivan JP (2018). A critical review of consumer wearables, mobile applications, and equipment for providing biofeedback, monitoring stress, and sleep in physically active populations. *Frontiers in Physiology* 9:743.
4. Billman GE (2013). The LF/HF ratio does not accurately measure cardiac sympatho-vagal balance. *Frontiers in Physiology* 4:26.
5. Impellizzeri FM, Tenen MS, Kempton T, Novak A, Coutts AJ (2020). Acute:chronic workload ratio: conceptual issues and fundamental pitfalls. *International Journal of Sports Physiology and Performance* 15(6):907–913.
6. Phenomnomnominal (Craig Spence). betterer: incremental improvement test runner. Open-source project, github.com/phenomnomnominal/betterer.
7. Mitchell M, Wu S, Zaldivar A, Barnes P, Vasserman L, Hutchinson B, Spitzer E, Raji ID, Gebru T (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, 220–229.
8. Zaharia M, Chen A, Davidson A, et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin* 41(4):39–45.
9. Coles H, Laurent T, Henard C, Papadakis M, Ventresque A (2016). PIT: a practical mutation testing tool for Java. *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016)*, tool demonstration, 449–452.
10. Stryker Mutator. Stryker documentation: mutation score and thresholds. stryker-mutator.io.
11. IEC 62304:2006+AMD1:2015. Medical device software — Software life cycle processes. International Electrotechnical Commission.
12. ISO 13485:2016. Medical devices — Quality management systems — Requirements for regulatory purposes. International Organization for Standardization.
13. Jia Y, Harman M (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37(5):649–678.
14. Petrović G, Ivanković M (2018). State of mutation testing at Google. *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 163–171.
15. Preston-Werner T. Semantic Versioning 2.0.0. semver.org.
16. ISO 14971:2019. Medical devices — Application of risk management to medical devices. International Organization for Standardization.
17. Sculley D, Holt G, Golovin D, et al. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*.
18. Breck E, Cai S, Nielsen E, Salib M, Sculley D (2017). The ML test score: a rubric for ML production readiness and technical debt reduction. *IEEE International Conference on Big Data*, 1123–1132.
19. Megapixel99 (GitHub user). canfail: "Break the thing on purpose and check that your check notices." Python package, version 0.1.0 released on PyPI 2026-08-31. pypi.org/project/canfail; source github.com/Megapixel99/canfail. Accessed 2026-10-08.
20. Megapixel99 (GitHub user). didrun: "An exit code cannot tell you whether anything happened." Python package, version 0.1.0 released on PyPI 2026-08-31 (also on npm as @megapixel99/didrun). pypi.org/project/didrun; source github.com/Megapixel99/didrun. Accessed 2026-10-08.
21. Megapixel99 (GitHub user). restore-verified: "Temporarily modify a file, survive the signal, and prove the tree came back." Python package, version 0.0.1 released on PyPI 2026-08-31 (also on npm). pypi.org/project/restore-verified; source github.com/Megapixel99/restore-verified. Accessed 2026-10-08.

22. useblocks. sphinx-needs: Sphinx extension for requirements and specification items (configuration options `needs_statuses` and field schema validation). Open-source project, github.com/useblocks/sphinx-needs. Accessed 2026-10-08.
23. reqcov contributors. reqcov: requirements coverage for pull requests. Python package, version 0.1.0 released on PyPI 2026-09-04. pypi.org/project/reqcov/; source github.com/Antoine005/reqcov. Accessed 2026-10-08.
24. Doorstop: requirements management using version control. Open-source project, github.com/doorstop-dev/doorstop. Accessed 2026-10-08 (seen through a search-engine extract only).
25. CoverageRatchet: per-file coverage enforcement with automatic threshold ratcheting from Cobertura XML. NuGet package, owner michaelglass; versions listed from 2026-05-05. nuget.org/packages/CoverageRatchet. Accessed 2026-10-08.
26. markis (GitHub user). jest-ratchet: "Ratchet up code coverage — keep test coverage going only one direction — up". npm package, first published 2018-01-15. npmjs.com/package/jest-ratchet; source github.com/markis/jest-ratchet. Accessed 2026-10-08.
27. maiconsouza89/agents-skills, GitHub issue 130 (opened 2026-09-17): CI check failing when hashed content changes without a semantic-version bump. github.com/maiconsouza89/agents-skills/issues/130. Accessed 2026-10-08.
28. pandas-studio/agent-team-plugins, GitHub pull request 165 (opened 2026-09-27): CI check failing when hashed content changes without a semantic-version bump. github.com/pandas-studio/agent-team-plugins/pull/165. Accessed 2026-10-08.
29. Semgrep. Testing rules (`ruleid:` and `ok:` annotations). semgrep.dev/docs/writing-rules/testing-rules. Accessed 2026-10-08 (seen through a search-engine extract only).
30. Sharp C (2026). Independent validation of Emuqu's beat-interval methods against PhysioNet recordings. Emuqu repository, Tools/validation/RESULTS.md, run 2026-10-08. PhysioNet data: slpdb 1.0.0, capsldb 1.0.0, mitdb 1.0.0, nsr2db 1.0.0.
31. Sharp C. ChatGPT user messages, 2025-02-22 15:44:18 UTC onward, and the author's account of 2026-09-29; excerpts recovered through conversation-history retrieval on 2026-10-08; full transcript and stable conversation ID not exposed.

Appendix A: Parameter Table

Name	Value	Unit	Role
Hashed source files	2	files	Recovery-score constants file and HRV thresholds file
Hashed constants	155 (91 + 64)	declarations	Count at this revision
Constant regex	<code>static let\s+(\w+)\s*:\?s*\[\\w<>\\[\],.]*=\s*([-\d.]+)</code>	—	Extracts symbol and literal value
Comment strip (hash)	<code>//[^\n]* → empty</code>	—	Line comments only
Hash line format	<code><path>:<symbol>=<value>\n</code>	UTF-8	One per constant, per-file sorted
Digest	SHA-256, first 16 hex chars	64 bits	Constants fingerprint

Name	Value	Unit	Role
Baseline line	<version> <hash>	—	Recorded pair
Current version	v3.1.oct2026	—	Version this build computes
Legacy sentinel	unversioned	—	Decoded value for unstamped records
Ranking-key identifier	rankingWeight	—	Allowed only in the cause-detection directory
Register entries	19	entries	Count at this revision
Status vocabulary	validated, supported-transfer, awaiting-validation, scheduled-for-removal	—	Keys of <code>_status_meanings</code>
Required non-empty fields	affects, evidence, allowed_language	—	Per entry
Retracted strings	3	strings	Scanned in entries, app sources and docs (.swift, .md)
Discovery suffixes	Adjustment, Penalty, Bonus, Modifier, Damper, Fraction	—	Function-name shape
Discovery exclusions	names ending Descriptions or Message; cause-detection directory	—	Not score inputs
Discovered functions	13 (12 claimed, 1 waived)	functions	At this revision
Gate scripts	45	scripts	<code>check_*</code> and <code>enforce_*</code>
Planted violations	64	plants	Planted-violation harness
Plant exemptions	coverage-floor gate, Thread Sanitizer gate	—	Need a build or simulator
Targeted test mutations	125	mutations	Test-mutation harness
Build-failure retry	1	retry	Baseline suite run only
Wiring exemptions	1	gate	Coverage gate runs inside the coverage step
Preflight-scanned scripts	53	scripts	All shell scripts except the preflight gate
Gate exit codes	0 clean, 1 violation, 2 configuration error	—	Scoring, register, wiring gates
Helper exit code	70 (EX_OSERR)	—	Temp-file or empty-input failure
Lock-held exit code	75 (EX_TEMPFAIL)	—	Another mutator is live
Trailer	budget-raise-ok: <path> <reason>	—	Acknowledged loosening; case-insensitive key
Floor prefix	min_	—	Budget may only rise
Require-baseline flag	BUDGET_REQUIRE_BASELINE	env var	Missing baseline becomes a failure

Name	Value	Unit	Role
Doc-quote number regex	<code>(?![\w.]) (\d{2,6}) (?![\w.%])</code>	—	Bare number near a budget filename
Doc-quote window	current line + next line	lines	Where the number is sought
macOS billing multiplier	10	× Linux	CI cost basis
Full macOS run	~108 wall / ~1,080 billable	minutes	Measured by the repository

Appendix B: Pseudocode

B.1 Constants-hash version lock

```
function scoring_governance():
    problems = []
    for file in all_swift_files(APP_SOURCES):
        if file not under CAUSE_DETECTION_DIR:
            for (n, line) in lines(file):
                if "rankingWeight" in line: problems.add(file, n)
    h = sha256()
    count = 0
    for path in [RECOVERY_CONSTANTS, HRV_THRESHOLDS]:           # fixed order
        if not exists(path): exit 2
        src = regex_replace(read(path), "//[^\n]*", "")
        pairs = sorted(regex_findall(src, CONST_RE))             # (symbol, value)
        for (sym, val) in pairs:
            h.update(utf8(path + ":" + sym + "=" + val + "\n"))
            count += 1
    current_hash = hex(h)[0:16]
    version = regex_group(read(VERSION_FILE), 'static let current\s*=\s*"([^\n]+)"') or
exit 2
if not exists(BASELINE): exit 2
(rec_version, rec_hash) = split_whitespace(read(BASELINE))
if current_hash != rec_hash and version == rec_version:
    problems.add("constants changed without a version change")
print(count, current_hash, version)
exit 1 if problems else 0

decode ScoreBreakdown(record):
    scoringVersion = record.get("scoringVersion") ?? "unversioned"
```

B.2 Science register gate

```

function science_register():
    reg = parse_json(REGISTER) or exit 2
    VALID = keys(reg._status_meanings); if empty: exit 2
    problems = []; seen = {}
    for e in reg.entries:
        if e.id in seen: problems.add(dup)
        seen.add(e.id)
        if e.status not in VALID: problems.add(status)
        for f in [affects, evidence, allowed_language]: if empty(e[f]): problems.add(f)
        for t in e.tests or []: if not exists(t): problems.add(t)
    if reg.scoring_version != read_current_version(): problems.add(version)
    for c in reg._retracted_claims.claims:
        for e in reg.entries, f in [evidence, affects, allowed_language, notes]:
            if c in join(e[f]): problems.add(c)
        for file in files(APP_SOURCES, DOCS) with ext in {swift, md}:
            if c in read(file): problems.add(file, c)
    discovered = {}
    for file in swift_files(ANALYSIS_DIR) not under CAUSE_DETECTION_DIR:
        for name in findall("func\s+([a-zA-Z]+
(?:Adjustment|Penalty|Bonus|Modifier|Damper|Fraction))\b"):
            if not name.endswith("Descriptions" or "Message"): discovered.add(name)
    claimed = union(e.symbols for e in entries)
    waived = {k in reg._waived_symbols if not k.startswith("_")}
    for n in discovered - claimed - waived: problems.add(unclassified n)
    for n in (claimed u waived) - discovered: problems.add(stale n)
    print tally
    if problems: exit 1
    if len(entries) < read_int(REGISTER_FLOOR): exit 1
    exit 0

```

B.3 Planted-violation harness

```

for (name, gate, file, mutation) in PLANTS:
    if run(gate) != 0: SKIP; continue
    backup(file)
    if not run_python(mutation): restore(); SKIP; continue
    code = run(gate); restore_all()
    if code != 0: PASS else: FAIL
    unplanted = [g for g in glob(check_*.sh, enforce_*.sh)
        if g != "check_gate_preflight.sh" and g not in NEEDS_BUILD
        and basename(g) not in read(this_script)]
    if unplanted: exit 1
    exit 1 if FAIL > 0 else 0

```

B.4 Preflight and wiring

```

preflight:
  for s in shell_scripts except self:
    for line in non_comment_lines(s):
      if matches("(?!gate_)\bmktemp\b") and "gate_mktemp" not in line: fail
      if "gate_mktemp" in line and "|| exit" not in line: fail
      if line sources "$(dirname "$0")/X" and not exists(scripts/X): fail

wiring:
  wf = read(WORKFLOW) or exit 2
  for g in gates (check_*/enforce_*) not in NOT_A_STEP:
    if basename(g) not substring of wf: unwired.add(g)
  exit 1 if unwired else 0

```

B.5 Targeted test mutations and leak detector

```

run_suite(s): xcodebuild test -only-testing s
  -> 0 passed | 2 if build banner or no "Test Suite|Test Case" | 1 otherwise
for (name, suite, file, mutation) in MUTATIONS:
  b = run_suite(suite); if b == 2: b = run_suite(suite)
  if b != 0: SKIP; continue
  backup(file); if not apply(mutation): restore(); SKIP; continue
  r = run_suite(suite); restore()
  if r != 0: PASS else: FAIL
exit 1 if FAIL or SKIP else 0

leak_detector:
  for case in split(read(MUTATION_SCRIPT), '\nmutate ')[1:]:
    path = first double-quoted app-directory .swift path in case
    orig = group(o='(.*)';assert) or group(s.count('(.*)')\s*[\>=<])
    if not path or not orig: unparsed.add; continue
    orig = unescape(orig); orig = resolve("'" + chr(N) + '" -> char N)
    if "chr(" in orig or "'" + " in orig: unparsed.add; continue
    if not exists(path) or orig not in read(path): leaked.add
  exit 1 if leaked or unparsed else 0

tree_lock_acquire(owner):
  if mkdir(LOCK) fails:
    if pid_alive(read(LOCK/pid)): exit 75
    warn "stale lock: previous run never restored"; rm LOCK; mkdir(LOCK) or exit 70
  write LOCK/owner, LOCK/pid

```

B.6 Direction-aware ratchet

```

base = arg or ("origin/"+GITHUB_BASE_REF) or BUDGET_BASE_REF
if not base or base == 40 zeros or not resolvable(base):
    if BUDGET_REQUIRE_BASELINE: exit 1 else: exit 0
acks = [path for msg in commit_messages(base..HEAD)
        for path in regex_i("^budget-raise-ok:\s*(\S+)", msg)]
for f in sorted(CI_DIR/*.txt):
    cur = int_or_skip(read(f)); prev = int_or_skip(git_show(base, f)) or skip
    if cur == prev: continue
    loosened = (cur < prev) if basename(f).startswith("min_") else (cur > prev)
    if loosened and f not in acks: violations.add(f, prev, cur)
exit 1 if violations else 0

count_for_budget(src):
    src = blank_block_comments_keep_newlines(src)
    src = blank_line_comments_same_length(src)
    src = blank(PLATFORM_SINGLETON.shared)
    return count(pattern, src)

```

Appendix C: Source Map

Method	Repository paths
Constants-hash lock, ranking- weight containment	scripts/check_scoring_governance.sh, .ci/scoring_constants_hash.txt (baseline), Emuqu/Sources/Utilities/Constants+RecoveryScore.swift, Emuqu/Sources/Analysis/HRVThresholds.swift
Version string and sentinel	Emuqu/Sources/Analysis/ScoringVersion.swift
Version persisted on breakdowns	Emuqu/Sources/Analysis/RecoveryScoreCalculator.swift, Emuqu/Sources/Analysis/VitalsScoring.swift, Emuqu/Sources/Storage/CloudKitSyncSupport.swift
Version shown	Emuqu/Sources/Views/RecoveryScoreDetailView+Sections.swift
Version given to the assistant	Emuqu/Sources/Assistant/Facts/AppFactResolver.swift, Emuqu/Sources/Assistant/Context/ContextBuilder.swift
Composite weights (governed by the register)	Emuqu/Sources/Utilities/Constants.swift
Science register and gate	Tools/science_register/register.json, scripts/check_science_register.sh, register floor file in .ci/
Copy perimeter (White Paper 3)	Tools/copy_linter/lint.py, Tools/copy_linter/prohibited_terms.json, scripts/check_perimeter_sync.sh
Planted violations	scripts/verify_gates_fail.sh
Fail-closed helpers and tree lock	scripts/lib/preflight.sh, scripts/check_gate_preflight.sh
Wiring	scripts/check_gates_wired.sh, .github/workflows/ci.yml

Method	Repository paths
Targeted test mutations and leak backstop	<code>scripts/verify_tests_fail.sh</code> , <code>scripts/check_no_leaked_mutations.sh</code>
Ratchets	<code>docs/adr/004-ratcheted-budgets.md</code> , <code>scripts/check_budget_monotonicity.sh</code> , <code>scripts/enforce_tech_debt_budgets.sh</code> , <code>scripts/enforce_swifflint_budget.sh</code> , budget files in <code>.ci/</code>
Documentation constants rule	<code>scripts/check_doc_links.sh</code>
CI posture and evidence	<code>docs/CI_POSTURE.md</code> , <code>.github/workflows/gates.yml</code> , <code>.github/workflows/ci.yml</code> , <code>scripts/evidence_report.sh</code> , <code>Makefile</code>